



Universidad Nacional Autónoma de México

Faculty of Engineering

CRYPTOGRAPHY TEAM 4

Team Members:

319215555 - Alfaro Dominguez Arturo

422526115- Herrera Gallegos Karina Jezuleth

422526067 - Sanchez Duran Dannae

422534286 - Infante Martinez Javier de Jesus

Gpo. 4 Semester: 2027-1

Deadline:08/11/2026

D5 Signatures & Authentication

---

1. Introduction

In previous installments, we developed a system to encrypt files and securely share them among different users. To achieve this, we implemented AES-GCM in D2 and later incorporated hybrid encryption using ECIES in D3 (Updated).

These features protect file confidentiality and allow you to specify which recipients can retrieve their content. However, a limitation remained: the system lacked a cryptographic mechanism to verify who signed a file or detect modifications to a digital signature.

For this reason, in D5, we incorporated digital signatures using the Ed25519 algorithm. This implementation allows you to verify the authenticity of the source using a trusted public key and confirm that the container has not been modified after being signed.

2. Objective

- Implement a digital signature mechanism that provides origin authentication and integrity verification for encrypted files on our system.
-
- Specifically, we aim to:
- Generate digital signatures using Ed25519 private keys.
- Verify signatures using the corresponding public keys.
- Protect both the encrypted content and the container metadata.
- Reject files with invalid, missing, or modified signatures.
- Integrate these functionalities without altering the cryptographic operation developed in D2 and D3.

3. Digital signature design

3.1 Selected Algorithm

We selected Ed25519 to implement digital signatures. Ed25519 uses public-key cryptography. The sender generates a signature with their private key, and the recipient can verify it using the corresponding public key.

In our implementation, the signature keys are independent of the ECIES keys used for file sharing. This separation allows us to maintain distinct cryptographic functions for encryption and authentication.

3.2 Key Generation

We implemented the function `generate_signature_key_pair()`, which generates an Ed25519 private key from which its public key is derived.

We also incorporated functions for importing and exporting keys. Private keys are stored in PEM PKCS#8 format with password protection, while public keys use the PEM format `SubjectPublicKeyInfo`.

3.3 Data Included in the Signature

We decided to sign the entire encrypted container, excluding only the signature field, as it does not yet exist at the time of signature calculation. The protected data includes:

1. Ciphertext.
2. Nonce used during encryption.
3. Authentication tag.
4. File metadata.
5. List of recipients and their encrypted keys.
6. Signer ID.
7. Signature algorithm.

To avoid differences in data representation, we use canonical JSON serialization, which orders the keys and establishes a consistent format.

This ensures that any modification of the signed fields produces a different message and, therefore, invalidates the original signature.

3.4 Hash → Sign → Verify Process

The implemented procedure follows the three requested operations.

Hash: First, a canonical representation of the container is constructed, and a protocol context identifier is added. Then, its hash is calculated using SHA-256.

Sign: The resulting hash is signed using the sender's private key, Ed25519. Verify: The recipient recalculates the digest and verifies the signature with the expected sender's public key.

Although Ed25519 can sign messages directly, in this instance we applied SHA-256 beforehand to follow the scheme requested by the professor. The additional context allows us to distinguish our protocol's signatures from those used in other applications.

4. Identification and storage of the signature

To identify the signer, we use the signer_id field, calculated using SHA-256 on the DER SubjectPublicKeyInfo representation of their public key Ed25519.

The generated signature is 64 bytes long and is encoded using Base64 for storage within the JSON container.

- The container includes three additional fields:
- signer_id: a fingerprint of the signer's public key.
- signature algorithm: identifies the algorithm used.
- signature: stores the encoded digital signature.

It is important to note that the identifier alone does not prove a person's identity. To trust the origin, the recipient must first have an authentic public key from the expected sender.

5. Integration with D2 and D3

The D5 implementation was developed using existing functionalities, without directly modifying the cryptographic modules of previous releases.

The procedure for generating a signed file is as follows:

1. The system receives the original file.
2. Its content is encrypted using AES-GCM.
3. The file keys are protected for authorized recipients using ECIES.
4. The container is created with the encrypted content and its metadata.
5. The hash of the data to be signed is calculated.
6. The digital signature is generated using Ed25519.
7. The signed container is stored with the .vault extension.

This procedure corresponds to the requested workflow:

Encrypt → Add metadata → Sign → Store

6. Verification and decryption process

To recover a signed file, the system must verify its authenticity before using the recipient's private key for decryption.

1. The procedure is as follows:
2. Read the .vault container.
3. Extract the data necessary to verify the signature.
4. Verify the signature using the expected signer's public key.
5. If the signature is valid, validate the container's structure.
6. Proceed with decryption using the mechanism developed in D3.
7. If the signature is invalid, reject the file without attempting decryption.

This sequence is important because it prevents cryptographically processing a file whose origin or integrity has not been verified.

7. Security decisions

7.1 Why do we sign the ciphertext and not the plaintext?

We chose to sign the encrypted content because it allows us to verify the authenticity and integrity of the container before decrypting it.

This way, the recipient can detect any modifications without needing to retrieve the original content first. Furthermore, the signature ensures that only the encrypted data being stored and shared is protected.

7.2 What would happen if we didn't verify the signature first?

If the system attempted to decrypt before verifying, it would be processing a container whose origin has not yet been authenticated.

This could allow manipulated files to reach later stages of processing. Therefore, we mandated signature verification before decryption in the D5 flow.

7.3 What would happen if we excluded the metadata?

If we only signed the encrypted content, some fields in the container could be modified without invalidating the signature.

For example, an attacker could try to alter the recipient list, identifiers, or file context.

To prevent this, we include all container fields in the signature calculation, except for the digital signature itself.

7.4 Key Protection

Ed25519 private keys are used exclusively for signing and must be kept secure. Public keys are used for verification, but it is necessary to know beforehand which one corresponds to the

expected sender. Our system uses cryptographic fingerprints to identify them; it does not implement a full digital certificate infrastructure.

8. Tests performed

To verify the functionality of D5, we developed unit and integration tests using pytest.

The tests cover the five main scenarios.

Test	Expected Result
Valid Signature	File Accepted
Encrypted Content Modified	File Rejected
Metadata Modified	File Rejected
Incorrect Public Key	File Rejected
Signature Removed	File Rejected

We also incorporated additional scenarios to verify the rejection of unauthorized recipients and to ensure that the signature is validated before decryption.

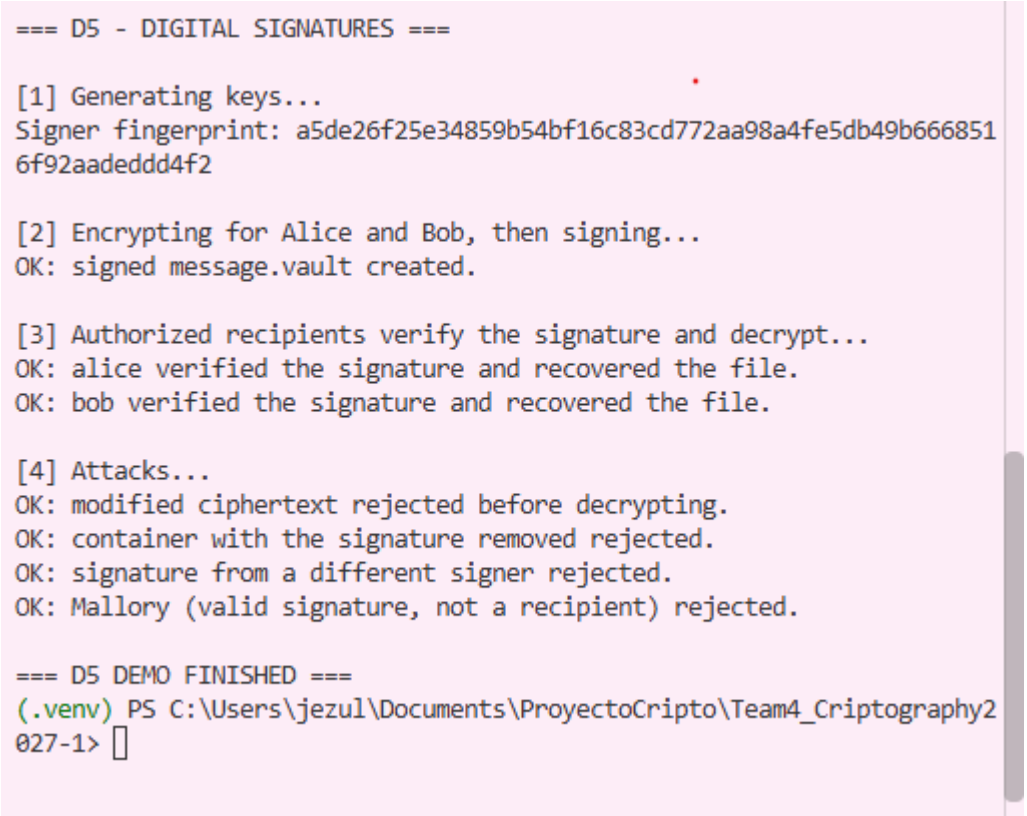
Running the complete project suite yielded the following results:

```
027-1> python -m pytest -q
........................................................... [ 27%]
........................................................... [ 54%]
........................................................... [ 81%]
.........................................                  [100%]
227 passed in 2.58s
```

9. Demonstration of operation

We performed a demonstration using the demo_d5.py file.

1. First, we generated the necessary keys and encrypted a file for two authorized recipients: Alice and Bob. Then, we signed the container using Ed25519.
2. During recovery, both Alice and Bob successfully verified the signature and were able to decrypt the file.
3. Afterward, we performed several manipulation attempts:
4. Modification of the encrypted content.
5. Deletion of the signature.
6. Use of a signature belonging to another signer.
7. Attempt at access using Mallory, who was not an authorized recipient.



10. Conclusions

In this release, we incorporated digital signatures into our secure file sharing system, complementing the symmetric and hybrid encryption functionalities developed previously.

With Ed25519, we successfully implemented a mechanism that allows us to verify the sender's signature and detect modifications to the encrypted content or the container's metadata.

One of the most important aspects was establishing signature verification as a mandatory step before decryption. This allows us to reject manipulated files before processing them further.

Automated testing and demonstrations allowed us to verify the integration's functionality, including valid cases and various alteration attempts.

Ultimately, we understood that confidentiality, integrity, and authentication are distinct security properties that must be implemented through appropriate mechanisms. Although our project does not yet incorporate a complete digital certificate infrastructure, D5 represents a significant step forward in building a more secure file sharing system.

11. References

Introduction. (s. f.). https://ed25519.cr.yp.to/

Reys, G. (1d. C., enero 1). *definition of ed25519*. Unix Tutorial.

https://www.unixtutorial.org/glossary/ed25519/